

```

import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib
matplotlib.use("Agg")
from wheelchair_env import WheelchairNavEnv

# — Seeds —————
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
model_path = "sarsa/sarsa_wheelchair_converged_v4.pt"

class SARSAAgent:
    def __init__(self, state_dim=7, n_actions=5, hidden=256):
        self.device = "cuda" if torch.cuda.is_available() else "cpu"
        self.n_actions = n_actions
        self.epsilon = 0.0 # greedy at eval time
        self.eps_min = 0.0
        self.eps_decay = 1.0
        self.q_net = nn.Sequential(
            nn.Linear(state_dim, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, n_actions),
        ).to(self.device)

    def q_values(self, state):
        s = torch.FloatTensor(state).unsqueeze(0).to(self.device)
        with torch.no_grad():
            return self.q_net(s).cpu().numpy().flatten()

    def predict(self, obs, deterministic=True):
        """Same interface as SB3's model.predict() for easy drop-in."""
        q_vals = self.q_values(obs)
        action = int(np.argmax(q_vals))
        return action, None

    def load(self, path):
        ckpt = torch.load(path, map_location=self.device)
        self.q_net.load_state_dict(ckpt["q_net"])
        self.epsilon = ckpt.get("epsilon", 0.0)
        print(f" Loaded ← {path} (ε={self.epsilon:.4f})")

# — Load model —————
model = SARSAAgent(state_dim=7, n_actions=5, hidden=256)
model.load(model_path)
model.epsilon = 0.0 # force greedy – no exploration at eval time

# — Env —————
env = WheelchairNavEnv(
    action_type='discrete', # ← SARSA uses discrete, not continuous
    n_people=3,
    n_obstacles=4,
    render_mode='rgb_array',
    seed=SEED
)

# — Eval loop (unchanged from before) —————
N_EVAL = 1000
eval_rewards, eval_success, eval_collisions = [], [], []

for ep in range(N_EVAL):
    obs, _ = env.reset(seed=SEED + ep)
    total_reward = 0.0
    done = False

    while not done:
        action, _ = model.predict(obs, deterministic=True)
        obs, reward, terminated, truncated, info = env.step(action)
        total_reward += reward
        done = terminated or truncated

    dist = info.get("dist_to_goal", float("inf"))
    success = int(terminated and dist <= 0.4)
    collision = int(terminated and dist > 0.4)

    eval_rewards.append(total_reward)
    eval_success.append(success)

```

```

eval_success.append(success)
eval_collisions.append(collision)

print(f"Ep {ep+1:>3} | reward: {total_reward:>8.2f} | {'GOAL' if success else 'COLLISION' if collision else 'TIMEOUT'}")

print(f"  Model path      : {model_path}")
print(f"  Total reward      : {np.sum(eval_rewards):.2f}")
print(f"  Average reward    : {np.mean(eval_rewards):.2f}")
print(f"  Success rate      : {np.mean(eval_success)*100:.2f} %")
print(f"  Collision rate    : {np.mean(eval_collisions)*100:.2f} %")

```

```

Loaded ← sarsa/sarsa_wheelchair_converged_v4.pt  (ε=0.0000)
Ep  1 | reward:  143.07 | GOAL
Ep  2 | reward:  120.78 | GOAL
Ep  3 | reward:  194.19 | GOAL
Ep  4 | reward:  -38.76 | GOAL
Ep  5 | reward:  248.07 | GOAL
Ep  6 | reward:  111.79 | GOAL
Ep  7 | reward: -141.62 | COLLISION
Ep  8 | reward:  124.28 | GOAL
Ep  9 | reward:   71.48 | GOAL
Ep 10 | reward:  -75.35 | COLLISION
Ep 11 | reward: -83.97 | COLLISION
Ep 12 | reward:  152.81 | GOAL
Ep 13 | reward:  256.67 | GOAL
Ep 14 | reward:  238.58 | GOAL
Ep 15 | reward:  215.20 | GOAL
Ep 16 | reward:   98.04 | GOAL
Ep 17 | reward:  277.04 | GOAL
Ep 18 | reward:  229.82 | GOAL
Ep 19 | reward:   46.11 | GOAL
Ep 20 | reward: -93.41 | COLLISION
Ep 21 | reward:   23.64 | GOAL
Ep 22 | reward:   30.03 | GOAL
Ep 23 | reward:  189.35 | GOAL
Ep 24 | reward:  230.41 | GOAL
Ep 25 | reward:   82.41 | GOAL
Ep 26 | reward:   25.91 | GOAL
Ep 27 | reward:  107.17 | GOAL
Ep 28 | reward:   81.92 | GOAL
Ep 29 | reward: -68.33 | GOAL
Ep 30 | reward:  290.94 | GOAL
Ep 31 | reward:   49.86 | TIMEOUT
Ep 32 | reward:    6.10 | GOAL
Ep 33 | reward:   78.59 | GOAL
Ep 34 | reward:  153.75 | GOAL
Ep 35 | reward: -429.61 | GOAL
Ep 36 | reward:   86.23 | COLLISION
Ep 37 | reward:   79.74 | GOAL
Ep 38 | reward:   84.88 | GOAL
Ep 39 | reward:   46.81 | GOAL
Ep 40 | reward:   22.20 | COLLISION
Ep 41 | reward:   71.81 | COLLISION
Ep 42 | reward:  225.14 | GOAL
Ep 43 | reward:  -2.11 | COLLISION
Ep 44 | reward:  282.97 | GOAL
Ep 45 | reward:  118.72 | COLLISION
Ep 46 | reward: -299.70 | GOAL
Ep 47 | reward:  -84.14 | GOAL
Ep 48 | reward:  -20.52 | GOAL
Ep 49 | reward:  224.70 | GOAL
Ep 50 | reward:   27.77 | GOAL
Ep 51 | reward: -117.82 | GOAL
Ep 52 | reward:  -84.88 | GOAL
Ep 53 | reward: -151.84 | GOAL
Ep 54 | reward:  124.25 | GOAL
Ep 55 | reward:  220.28 | GOAL
Ep 56 | reward:    6.91 | GOAL
Ep 57 | reward:  248.28 | GOAL

```

Start coding or generate with AI.

